Nama : Averose Arthur R

TI-1H/04/254107020042

Prakt ASD

Jobsheet 5 divide and conq

5.1.1. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

   - Bagian if: Berfungsi sebagai base case (kasus dasar). Dalam kode faktorial, if (n == 1) digunakan untuk menghentikan rekursi saat nilai n sudah mencapai 1, karena faktorial dari 1 adalah 1.

   - Bagian `else`: Berfungsi sebagai recursive step. masalah besar dipecah menjadi sub-masalah yang lebih kecil (n-1) sampai mendekati base case.

2. Apakah memungkinkan perulangan pada method faktorialBF()diubah selain menggunakan for? Buktikan!

   - Ya, sangat memungkinkan. Kita bisa menggunakan perulangan `while` atau `do-while`.

   - Bukti (menggunakan `while`):

```
int faktorialIBF(int n){
    int fakto = 1;
    for (int i = 1; i <= n; i++) {
        fakto = fakto * i;
    }
    return fakto;
}
```

3. Jelaskan perbedaan antara fakto *= i;dan int fakto = n * faktorialDC(n-1);!

- `fakto *= i;`: cara **iteratif** (Brute Force). Perhitungan dilakukan selangkah demi selangkah di dalam satu siklus perulangan yang sama. Variabel `fakto` langsung diupdate di setiap iterasi.

- `int fakto = n * faktorialDC(n-1);`: Merupakan cara **rekursif** (Divide and Conquer). Perhitungan tidak langsung selesai, melainkan program "menunggu" hasil dari pemanggilan fungsi `faktorialDC` berikutnya. Pemecahan masalah dilakukan dengan membagi angka n menjadi sub-masalah $(n-1)$.

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

- `faktorialBF()`: Bekerja secara linier dan langsung (Iteratif). Program melakukan instruksi pengulangan secara beruntun sampai kondisi berhenti tercapai. Biasanya lebih hemat memori karena tidak perlu menyimpan tumpukan pemanggilan fungsi (*stack*).

- `faktorialDC()`: Bekerja dengan membagi masalah menjadi masalah kecil yang serupa (Rekursif). Program memanggil dirinya sendiri. Cara ini lebih deklaratif dan seringkali lebih mudah dibaca untuk masalah yang kompleks, meskipun menggunakan lebih banyak memori untuk *call stack*.

5.1.2. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF()dan pangkatDC()!

- `pangkatBF()`: Menggunakan algoritma Brute Force dengan melakukan perkalian angka basis secara berulang sebanyak n kali di dalam sebuah perulangan (*loop*). Kompleksitasnya adalah linier O(n).

- `pangkatDC()`: Menggunakan algoritma Divide and Conquer dengan membagi pangkat menjadi dua bagian di setiap tahap rekursi. Jika pangkat genap, ia menghitung hasil dari setengah pangkatnya lalu dikuadratkan. Jika ganjil, ia melakukan hal yang sama lalu dikalikan dengan basisnya sekali lagi. Ini lebih efisien dengan kompleksitas O(\log n)$.

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!

- **Ya, sudah**. Tahap *combine* (penggabungan) dalam Divide and Conquer terjadi ketika hasil dari sub-masalah dikalikan kembali untuk mendapatkan hasil akhir.
- **Bukti pada kode**:
  - `return (pangkatDC(a, n/2) * pangkatDC(a, n/2));` (untuk pangkat genap)
  - `return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);` (untuk pangkat ganjil)

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

   Relevansi: Tetap relevan jika kita ingin method tersebut bersifat fleksibel (bisa menghitung angka apa saja tanpa bergantung pada status objek). Namun, jika mengikuti prinsip enkapsulasi objek, lebih baik menggunakan atribut yang sudah ada.

   Bisa dibuat tanpa parameter: Bisa, dengan cara mengakses atribut nilai dan pangkat menggunakan kata kunci this.

   Contoh kode:

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < this.pangkat; i++) {
        hasil = hasil * this.nilai;
    }
    return hasil;
  }
```

4. Tarik tentang cara kerja method pangkatBF()dan pangkatDC()!

- `pangkatBF()`: Bekerja dengan cara iteratif yang sederhana. Ia melakukan komputasi secara sekuensial (berurutan) dari awal sampai akhir sesuai jumlah pangkatnya.
- `pangkatDC()`: Bekerja secara rekursif dengan memecah masalah (pangkat) menjadi bagian-bagian yang lebih kecil secara logaritmik. Hal ini membuat proses perhitungan jauh lebih cepat untuk angka pangkat yang sangat besar dibandingkan cara Brute Force.

5.1.3. Pertanyaan

1. Kenapa dibutuhkan variable midpada method TotalDC()?

   - Variabel `mid` sangat penting karena berfungsi sebagai penentu titik tengah array. Dengan adanya `mid`, algoritma dapat membagi satu masalah besar (array utuh) menjadi dua sub-masalah yang lebih kecil (array bagian kiri dan array bagian kanan) secara terus-menerus hingga mencapai ukuran terkecil.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

   Statement tersebut merupakan tahap **Conquer**. Kode ini melakukan pemanggilan fungsi dirinya sendiri secara rekursif untuk menghitung total elemen di bagian kiri (indeks `l` sampai `mid`) yang disimpan dalam `lsum`, dan total elemen di bagian kanan (indeks `mid+1` sampai `r`) yang disimpan dalam `rsum`.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

   Ini merupakan tahap Combine (penggabungan). Setelah array dipecah hingga bagian

terkecil (base case) dan nilainya didapatkan, hasil penjumlahan dari sub-masalah bagian kiri dan kanan harus digabungkan kembali untuk menghasilkan total keuntungan keseluruhan perusahaan.

4. Apakah base case dari totalDC()?

```
if (l == r) {
    return arr[l];
}
```

Artinya, ketika indeks kiri (`l`) sama dengan indeks kanan (`r`), itu menandakan array hanya memiliki satu elemen, sehingga fungsi langsung mengembalikan nilai elemen tersebut tanpa melakukan pembagian lagi.

5. Tarik Kesimpulan tentang cara kerja totalDC()

- **Divide**: Membagi array menjadi dua bagian menggunakan variabel `mid`.
- **Conquer**: Menyelesaikan penjumlahan setiap bagian secara rekursif hingga mencapai satu elemen tunggal.
- **Combine**: Menjumlahkan kembali hasil dari setiap sub-bagian yang telah dihitung untuk mendapatkan nilai total akhir. Metode ini sangat efisien untuk pemrosesan data dalam skala besar dan memudahkan implementasi komputasi paralel.